

CODEV Recherche

Classification de données pour de l'apprentissage semi-supervisé

Sous la supervision de Bastien Pasdeloup et Mounia Hamidouche

Le *Machine Learning* a connu de nombreux changements et glissements ces dernières années. Si dans les imaginaires collectifs l'exemple le plus connu reste celui de l'algorithme en capacité de reconnaître des chiffres manuscrits, les intelligences artificielles modernes sont désormais aptes, pour ne citer que ces exemples, à détecter des objets ou bien à classer des images. C'est pour cette raison que le *Machine Learning* est aujourd'hui massivement utilisé dans le domaine de la *Data Science* et de l'analyse de données pour développer, tester et appliquer des algorithmes d'analyses prédictives. Aussi bien, la recherche de nouvelles techniques d'optimisation de ces algorithmes est-elle un enjeu contemporain majeur auquel essaie de répondre une communauté scientifique en développement.

Il convient de noter qu'un modèle de *Machine Learning* se définit comme la concaténation de deux ensembles qui peuvent, chacun, être améliorés : un extracteur et un classifieur. Le rôle de l'extracteur est de dessiner un réseau de neurones capable d'adapter ses connexions en fonction des entrées qui lui sont données, quand celui du classifieur consiste à trier et classer les données de manière efficace.

Pour ce projet, la tâche nous revient de traiter un problème dit de classification ou de *clustering* parce qu'il s'agit de déterminer la classe d'un objet en entrée du classifieur. C'est pourquoi notre étude consiste à améliorer le classifieur et non l'extracteur.

Par ailleurs, afin d'assurer leur bon fonctionnement, les algorithmes de *Machine Learning* ont besoin d'être entraînés pour fonctionner de manière satisfaisante. Il s'agit d'un apprentissage dit « supervisé » ce qui revient à intégrer uniquement des données labellisées dans l'algorithme. En réalité, l'ordinateur connaît la classe de chacun des attributs en entrée et se charge de comparer avec la classe attribuée en sortie de l'algorithme pour mesurer l'efficacité de ce dernier. Cependant, lorsque les jeux de données deviennent très importants – soit des millions de données différentes destinées à entraîner l'algorithme –, cette opération s'avère parfois très fastidieuse. C'est la raison pour laquelle l'apprentissage dit « semi-supervisé », lequel utilise à la fois des données étiquetées et des données non étiquetées – autrement appelées *features* –, peut être particulièrement stimulant. En effet, contrairement à l'apprentissage supervisé, le semi-supervisé requiert une quantité de données moindre et est en conséquence beaucoup moins coûteux.

En un mot, et pour problématiser en synthèse, l'objet de cette étude est de développer un classifieur de données dans le cadre de l'apprentissage semi-supervisé.

Pour ce faire, nous allons faire appel au Traitement du Signal sur Graphes (TSG). En effet, dans le cadre de l'apprentissage semi-supervisé, nous disposons de peu de données et par conséquent, le classifieur sera particulièrement sensible à chacune d'entre elles. C'est pour cette raison que « lisser » les données les rendraient ainsi plus cohérentes aux yeux du classifieur. Dans un premier mouvement, on pourrait penser qu'il suffit de filtrer les données en entrée de l'extracteur mais cette possibilité est vite écartée car la tâche est complexe dans le domaine de représentation initial des données : cela serait susceptible de limiter les performances du programme. En revanche, filtrer les données en sortie de l'extracteur s'avère utile pour le « débruitage » car l'algorithme pourra considérer des données plus abstraites. Dès lors, la force du TSG est évidente : elle a le pouvoir de représenter, d'une part, les données labellisées de chaque classe par les sommets d'un graphe et d'autre part, la similitude entre ces données par les poids dudit graphe. Il suffit enfin de filtrer chaque graphe.

Pour finir, notre projet a été encadré par Bastien Padeloup et Mounia Hamidouche, tous les deux enseignants-chercheurs à l'IMT Atlantique sur le campus de Brest. Ils nous ont suggéré d'implémenter, en plus du TSG décrit dans leur article *Improving Classification Accuracy with Graph Filtering*, un algorithme dit de *Mean Shift*. Ce dernier consiste à déplacer le centroïde de chacun des *clusters* de manière itérative vers une région de densité élevée jusqu'à obtenir une convergence. Dans le cas de notre étude, cela revient à évaluer un centroïde pour chaque classe – soit la « moyenne » des données labellisées de chacune des classes – puis à le déplacer en fonction des données non labellisées.

Notre conclusion se propose de questionner la compatibilité du TSG et du *Mean Shift*, cela en comparant nos résultats avec ceux obtenus par nos tuteurs.

I. Conception

Pour le projet, nous disposons d'un jeu de 20 classes différentes contenant chacune 600 *features*. Afin de nous placer dans le cadre de l'apprentissage semi-supervisé, sur les conseils de nos tuteurs, nous considérerons 5 classes choisies de manière aléatoire contenant chacune 5 données labellisées et 15 données non labellisées, en sorte que nous disposons de 100 données qui seront tirées aléatoirement. En outre, ces tirages seront réitérés à chaque fois que l'algorithme sera lancé.

Ainsi, l'objectif premier est de construire un centroïde pour chaque classe grâce aux données labellisées. Nous disposerons de cette façon d'un jeu de données appartenant de manière certaine à un *cluster* et les données non étiquetées serviront de mesure pour l'efficacité du programme. En effet, puisque l'algorithme devra affecter chaque donnée non labellisée à une classe, il suffira ensuite de vérifier s'il l'a affectée à la classe dans laquelle cette donnée avait été tirée aléatoirement. En l'occurrence, il est presque impossible d'arriver à une efficacité de 100%, mais nous misons sur une efficacité de 90%, dans le meilleur des cas.

A. Principes du Traitement du Signal sur Graphes :

Comme cela a été évoqué dans l'introduction, l'étude des données labellisées doit se faire après filtrage de ces dernières. Pour ce faire, nous devons étudier le formalisme du TSG. Prenons un graphe $G = \{V, E\}$ avec V l'ensemble des sommets et E l'ensemble des arrêtes. Mathématiquement, un graphe est représenté par sa matrice d'adjacence W avec :

$$W[i, j] = \begin{cases} w_{ij}, & \text{si l'arrête } (i, j) \in E \\ 0, & \text{sinon} \end{cases}$$

Avec : w_{ij} la valeur de l'arrête reliant les sommets i et j tel que $w_{ij} \in \mathbb{R}_{+}^{\mathbb{I} \times \mathbb{I}}$

À partir de la matrice d'adjacence, il est possible d'établir la matrice de degré du graphe définie comme tel :

$$D = \text{diag} \left(\left[\sum_{j=1}^{|V|} W[i, j] \right] \text{ avec } i \in \{1, \dots, |V|\} \right) \quad (1)$$

On peut alors définir le Laplacien du graphe G : $L = D - W$ (2)

Si tous les sommets du graphe G sont reliés, il est alors possible d'écrire le Laplacien normalisé de G :

$$L = I - D^{-\frac{1}{2}} W D^{-\frac{1}{2}} \quad (3)$$

Avec : I la matrice identité

La matrice W étant symétrique à valeurs réelles et les matrices I et D étant diagonales, on en déduit que L est elle-même symétrique à valeurs réelles. Ainsi, d'après le théorème spectral, elle peut être diagonalisée au moyen d'une matrice de passage orthogonale que l'on notera U . Cette dernière est composée des vecteurs propres de W et possède $\mathbb{I} V \mathbb{I}$ valeurs propres non nulles notées telles que :

$$\lambda_1 \leq \dots \leq \lambda_{|V|}$$

En notant Λ l'ensemble de ces valeurs propres, dès lors, on a :

$$L = U \text{diag}(\Lambda) U^T \quad (4)$$

On a diagonalisé L . On cherche désormais à filtrer les données étiquetées. Pour ce faire, il faut considérer chaque donnée labellisée comme un vecteur $x \in \mathbb{R}^{|V|}$. L'opération $\hat{x} = U^T x$ est appelée *Graph Fourier Transform* (GFT). Son inverse est donné par $x = U \hat{x}$ (iGFT).

Par analogie avec la transformée de Fourier classique qui consiste à passer un signal dans l'espace des fréquences, on peut considérer les valeurs propres du Laplacien comme des fréquences de ce dernier. On pourrait ainsi filtrer les « hautes fréquences » du Laplacien, ce qui reviendrait à réduire la variance des données, comme le montre le papier *Improving Classification Accuracy with Graph Filtering* écrit par nos tuteurs (cf. partie 3 : *Effect of low-pass graph filters on centroids*). En effet, retirer les hautes fréquences équivaut à supprimer le « bruit » et de fait, à conserver uniquement les fréquences d'intérêt du signal étudié. Nous devons par conséquent appliquer un filtre passe-bas sur le Laplacien. Pour ce faire, on note le filtre $h: \lambda \rightarrow h(\lambda)$ ce qui nous permet d'écrire la matrice de filtrage $H = \text{diag}(h(\lambda))$. Dès lors, un graphe dans le domaine de Fourier est donné par :

$$\hat{x} = U^T x \quad (5)$$

Par analogie avec la théorie de Fourier, on peut donc filtrer ce graphe grâce à la matrice H :

$$H \hat{x} = H U^T x \quad (6)$$

Il suffit enfin de multiplier (6) par U pour revenir dans le domaine des graphes d'après l'IGFT et ainsi obtenir x^{filter} :

$$x^{filter} = UH U^T x \quad (7)$$

Il est désormais nécessaire de transposer nos données labellisées en un graphe afin d'appliquer ce filtrage. Pour cela, on récupère les *features* $F = \phi(X)$ en sortie de l'extracteur, où X désigne les données en entrée de l'extracteur. On calcule ensuite la similarité entre chaque donnée labellisée d'une classe donnée puis on stocke chaque similarité dans une matrice. On a donc :

$$S_{i,j} = \begin{cases} s(F_{i,:}, F_{j,:}), \wedge \text{si } i \neq j \\ 0, \wedge \text{sinon} \end{cases}$$

Avec : $s:(x,y) \rightarrow s(x,y)$ la similarité entre x et y – typiquement une exponentielle décroissante ou un cosinus, tous deux fonctions de la norme entre les deux vecteurs.

On pourrait alors considérer la matrice de similarité comme la matrice d'adjacence du graphe d'une classe. Cependant, nous aurions un graphe complet et nous conserverions des similarités bien moins importantes entre certaines données étiquetées.

Aussi est-il pertinent de sélectionner seulement une partie des facteurs dans la matrice de similarité. On obtiendra de la sorte un graphe non nécessairement complet qui conservera les similarités les plus importantes. Ces dernières seront les données les plus « représentatives » de chaque classe.

Par conséquent, on décide de conserver les k plus grand facteurs de chaque ligne de la matrice de similarité et de nullifier les autres valeurs. Cette matrice sera considérée comme la matrice d'adjacence d'un graphe correspondant à une classe donnée. Il suffira ensuite d'appliquer le filtre comme en (7) :

$$F^{filter} = UH U^T F \quad (8)$$

Il vient maintenant d'expliquer le processus de l'algorithme du *Mean Shift* qui est essentiel dans la classification des données non étiquetées.

B. Algorithme du *Mean Shift* :

Les données labellisées ayant été traitées, il reste désormais à classer les données non labellisées grâce à l'algorithme du *Mean Shift*. D'après le processus décrit dans l'introduction, ce dernier doit affecter temporairement les *features* à une classe et permettre de réévaluer le centroïde de cette classe. En effet, il est nécessaire que l'algorithme puisse discuter de l'affectation d'un *feature* à une classe plutôt qu'à une autre. C'est pour cette raison qu'une donnée non étiquetée ne sera ajoutée de manière définitive à une classe – devenant ainsi étiquetée – que si elle est apparue plusieurs fois comme étant la plus proche d'un des centroïdes. Sans cette condition, elle ne se verra donc affectée que temporairement à la classe correspondante.

Cette partie de l'algorithme est fondamentale car elle différencie le *Mean Shift* de son homologue, l'algorithme des *K-means*, qui se satisfait d'ajouter des *features* aux *clusters* les plus proches. Le *Mean Shift* possède de ce fait un traitement dynamique dans la classification des

données non labellisées. En revanche, ce n'est pas le cas pour les centroïdes : le calcul de ces derniers ne sera pas remis en cause d'une itération à une autre.

Par ailleurs, il est essentiel que l'algorithme tienne compte de tous les points qui appartiennent déjà à une classe afin de réaliser une moyenne pondérée lors de la réévaluation des centroïdes et non pas une moyenne simple entre le centroïde et le *feature* ajouté temporairement.

Enfin, nous proposons de réaliser deux filtrages des données lorsqu'une donnée non étiquetée deviendra étiquetée. Premièrement, un filtrage de la classe à laquelle elle aura été ajoutée. Deuxièmement, un filtrage de toutes les données afin d'écarter les données par groupes. Plus précisément, cette partie du code doit permettre de rapprocher les données dans les zones de densité "élevées" et de les éloigner dans les zones de densité plus faibles. Aussi, ce double filtrage réalisé à chaque affectation devrait-il autoriser non seulement à rapprocher les données d'une même classe mais encore à éloigner celles n'appartenant pas à la même classe. Nous espérons de cette manière obtenir une efficacité plus explicite.

Maintenant que les fondations ont été posées, on peut en venir à notre programme qui est au cœur la réalisation de notre projet.

II. Réalisation

A. Extraction et tirage des données

Nous devons commencer par extraire les données. Nos tuteurs nous avaient mis à disposition un jeu de données que nous pourrions utiliser pour nos tests. Ainsi, une fois le module *pickle* importé et le chemin d'accès au fichier donné, on dispose de 20 classes – d'indices allant de 80 à 99 – de 600 vecteurs. Chaque vecteur représente une donnée sortie de l'extracteur : c'est pour cette raison que les coordonnées de ces vecteurs n'ont pas réalité physique.

Ensuite, il faut de tirer de manière aléatoire 5 classes pour ensuite piocher aléatoirement 20 données par classe : les 5 premières seront labellisées et les 15 autres, non labellisées. Ces opérations se font grâce au module *random* et la fonction utilisée est *sample (liste, nombre de tirages)* qui permet de faire un certain nombre de tirages sans remise. Procéder de cette façon permet d'éviter le croisement des données tout en conservant le caractère aléatoire du tirage au sort. Cette partie peut se faire grâce au *Code 1*.

B. Codage du TSG

Nous pouvons ensuite passer à la partie sur le TSG. Tout d'abord, nous devons établir la matrice de similarité. Pour ce faire, on crée une matrice carrée – comme toutes les autres matrices que nous serons amenés à traiter – nulle de dimension égale au nombre de features dont on doit évaluer la similarité. En l'occurrence, pour l'initialisation, étant donné que nous disposons de 5 données labellisées par classe, notre matrice de similarité sera de dimension 5x5. Il vient ensuite de parcourir cette matrice pour affecter la similarité entre une donnée *i* et une donnée *j*. Enfin, il est nécessaire de diviser par la norme L1 pour ainsi travailler avec des vecteurs de probabilités. Cela revient simplement à diviser par la somme des coordonnées de la matrice. Dès lors, on dispose du *Code 2*.

On remarquera que la similarité est une exponentielle décroissante de la norme écartant deux données labellisées, à un facteur près. Nous avons pensé à mettre cette distance au carré mais les résultats étaient moins concluants.

Une fois la matrice de similarité obtenue, nous devons extraire les k plus grands facteurs de chaque ligne. Pour cela, une fois une nouvelle matrice nulle définie, on parcourt chaque ligne de la matrice afin de les trier dans l'ordre décroissant dans une liste grâce à la commande *sorted* suivi de $[-1]$. Cette dernière commande permet de trier dans l'ordre décroissant alors que *sorted* trie dans l'ordre croissant. Grâce à cela, on ajoute les k premiers éléments de cette liste - qui sont aussi les k plus grands étant donné que la liste a été triée de manière décroissante - à une nouvelle liste. Aussi cette nouvelle liste permet-elle de voir si un élément de S fait partie des k plus grands facteurs. Si c'est le cas, il sera ajouté à la future matrice d'adjacence du graphe W . En l'occurrence, cette affectation de fait de manière symétrique puisque toute matrice d'adjacence est symétrique (cf. *Code 3*).

La matrice d'adjacence une fois créée, il suffit de filtrer le graphe correspondant. La première étape est d'obtenir le Laplacien normalisé $\bar{L}$. Ce dernier se calcule, d'après (3), à partir de trois matrices : la matrice d'adjacence W , la matrice identité I et la matrice de degré D à la puissance $-1/2$. Comme nous disposons déjà de la matrice d'adjacence, il vient de calculer I et D . La première s'obtient grâce au module *numpy* et plus précisément, grâce à la commande *np.eye(n)*, avec n la taille de W . La seconde est définie en (1). Elle correspond à la somme des poids des arrêtes voisines de tous les sommets du graphe. D'un point de vue matriciel, cela revient à sommer les coordonnées de chaque ligne et à les placer sur la diagonale de la matrice. Pour l'implémenter au code, il suffit simplement de parcourir tous les éléments d'une ligne i et à les sommer sur la colonne i de cette même ligne.

Une fois ces deux matrices obtenues, il vient diagonaliser $\bar{L}$. Cette opération peut se faire, une nouvelle fois, grâce au module *numpy* qui renseigne : un vecteur contenant les valeurs propres Λ - *np.linalg.eig(L)[0]* - et la matrice de passage orthogonale U permettant de passer dans la nouvelle base - *np.linalg.eig(L)[1]* -. Une fois que l'on a obtenu la matrice de passage orthogonale contenant les vecteurs propres (cf. I – Conception. Théorème spectral), il est nécessaire de transposer cette matrice afin de faire apparaître la diagonalisation. Dès lors, on vérifie bien l'équation (4).

Pour finir, le Laplacien étant dans une base adaptée aux valeurs propres – soit aux « fréquences » -, on peut alors appliquer le filtre comme décrit en (7). Il suffit pour cela de créer la matrice H qui est obtenue en parcourant chaque ligne i et d'appliquer le filtre à la valeur propre i , comme le montre la commande : $D[i,i]=\text{filtre}(vp[i])$. Une fois cela fait, il suffit de multiplier U , H , U^T et F pour obtenir le graphe filtré. Cette partie du code est donnée dans sa globalité dans le *Code 4*.

Afin que ces étapes puissent être appelées de manière plus concise dans le reste du code, on décide de créer une fonction *filter* qui crée la matrice de similarité, la transforme en matrice d'adjacence et qui applique le filtrage sur cette dernière. Elle retourne ainsi la classe filtrée, comme le montre le *Code 5*.

C. Programmation du *Mean Shift*

Maintenant que le filtrage a été explicité du point de vue de la programmation, il vient de mettre en place la fonction *predict* qui prédira la classe des données non labellisées. Cette fonction dispose de trois arguments : *classes* – soit la liste des données labellisées –, *features* – soit la liste des features – et le booléen *filtered* qui indique si l’on souhaite appliquer le TSG ou non. Rajouter ce booléen permettra par la suite de faire des tests comparatifs afin de juger la pertinence du TSG avec le *Mean Shift*. En outre, la fonction *predict* se divisera en deux étapes : l’initialisation des données labellisées et le *Mean Shift*.

La première étape se charge de filtrer ou non les classes et d’établir les centroïdes de chacune d’entre elles. Pour ce faire, il faut parcourir les différentes classes grâce à l’indice *c* qui prend de manière itérative la liste des données labellisées d’une classe. De plus, la commande *enumerate* permet de parcourir une liste correspondant au *range* de la classe considérée grâce à un indice *i*. En d’autres termes, pour première classe, *i* vaudra 0, pour la seconde classe, *i* vaudra 1, etc. Par ailleurs, le filtrage est appliqué avant le calcul des centroïdes grâce à *filter* si *filtered* prend la valeur *True*. Il vient ensuite d’évaluer les centroïdes en réalisant une moyenne des données labellisées grâce à la fonction *average* du module *numpy*. Enfin, on comptera le nombre de données ayant servi au calcul de chaque centroïde grâce à la liste *item_count_by_class*. On pourra alors réaliser des moyennes pondérées lors de la réévaluation des centroïdes et non pas une simple moyenne entre un *feature* et un centroïde. En l’occurrence, l’indice *i* prend tout son sens car il permet de mettre à jour l’*item_count_by_class* d’une classe donnée. Dès lors, on obtient le *Code 6*.

Une fois l’initialisation faite, on peut créer la matrice des distances des features aux centroïdes. Pour ce faire, il faut parcourir chaque feature et chaque centroïde évaluer la norme entre chacun d’entre eux. On initialise par ailleurs une matrice *deja_vu* dont les coordonnées sont des booléens tous initialisés à la valeur *False* (cf. *Code 7*). Cette matrice permettra de gérer l’affectation temporaire d’un feature à un centroïde et sera réévaluée à chaque itération. En effet, comme expliqué dans la partie I, un centroïde ne sera affecté de manière définitive à un cluster que s’il apparaît deux fois comme étant le plus proche du centroïde correspondant. Ainsi, lorsqu’un *feature j* sera affecté temporairement pour la première fois à une classe *i*, on aura *deja_vu[i, j]=True*.

D’autre part, il est évident qu’il faut programmer une façon de retirer les *features* affectés à une classe afin d’éviter de les reprendre en compte lors de l’actualisation des centroïdes. Pour ce faire, à chaque fois qu’un feature est affecté à un cluster, on le remplace par une liste vide et lorsqu’on réévalue la matrice des distances, on considère que tout élément vide de *features_copy* se trouve à une distance infinie des centroïdes grâce à la commande *Inf*.

A cela, on rajoute un critère d’arrêt cohérent avec ce remplacement de features par une liste vide. On indente une boucle *while* qui s’arrêtera lorsque *features_copy* ne contiendra que des listes vides, soit, en d’autres termes, lorsque tous les features auront été étiquetés.

Tous ces critères ayant été étayés, on peut maintenant passer au principe central du *Mean Shift* : l’affectation des données. Le principe est simple. On commence par parcourir les *features* un à un. Prenons un *feature j*. Si celui-ci est « vide », c’est qu’il a déjà été affecté à une classe ; on passe donc au feature suivant. On récupère alors l’indice du centroïde le plus proche – que l’on notera *i* – du *feature j*. Si *deja_vu[i, j]=True*, cela signifie qu’il faut ajouter *j* de manière définitive à la classe *i* et ajouter 1 à *item_count_by_class* pour ensuite calculer de manière cohérente le centroïde *i*. On réalise ensuite un filtrage de la classe ainsi mise à jour. Dans le cas contraire, autrement dit, si *deja_vu[i, j]=False* le *feature* n’a pas encore été affecté

temporairement au centroïde. On passe alors la valeur à *True* et on met à jour *item_count_by_class*. Pour finir, on actualise les centroïdes puis la matrice des distances et on recommence le processus tant que tous les *features* n'ont pas été affectés.

Tout le code est donné en annexe au *Code 8*.

D. Mesure de l'efficacité moyenne et de la variance des données

Il reste enfin à implémenter le code permettant de comparer les résultats donnés par la fonction *predict* avec la classe dans laquelle les *features* avaient été piochés. Pour ainsi dire, la fonction *predict* servait de test et il faut désormais mesurer son efficacité.